

An Agentic AI Framework with Large Language Models and Chain-of-Thought for UAV-Assisted Logistics Scheduling with Mobile Edge Computing

Hanwen Zhang[✉], *Graduate Student Member, IEEE*, Dusit Niyato[✉], *Fellow, IEEE*, Wei Zhang[✉], *Member, IEEE*, Xin Lou[✉], *Member, IEEE*, Malcolm Yoke Hean Low[✉], *Senior Member, IEEE*

Abstract—In cloud manufacturing, unmanned aerial vehicles (UAVs) can support both product collection and mobile edge computing (MEC). This joint operation forms a hybrid scheduling problem, where physical logistics decisions are coupled with computational task scheduling. In this paper, UAVs collect finished products from manufacturing stations and transport them back to a central depot. Meanwhile, computational tasks generated by industrial sensor devices at these stations are processed locally, at UAVs, or offloaded via UAVs to the cloud. This coupling makes the problem challenging. A UAV can provide MEC services only during its service window at a station, so routing decisions directly determine when UAV-assisted offloading is available. Routing decisions also affect the UAV energy budget and the availability of onboard computing and communication resources for computational task execution under task deadline constraints. To address this, we propose an agentic-AI-assisted optimization framework with two components. First, we develop an agentic AI that combines large language models, retrieval-augmented generation, and chain-of-thought reasoning to translate user input into an interpretable mathematical formulation for the hybrid scheduling problem. Second, we design a hierarchical deep reinforcement learning approach based on proximal policy optimization (PPO), where the upper layer learns UAV routing and the lower layer optimizes per-slot task execution and resource allocation. Simulation results show that the proposed framework yields more consistent formulations, while the hierarchical PPO

achieves full product collection in 99.6% of the last 500 episodes and maintains a 100% deadline satisfaction rate, with more stable performance than the advantage actor-critic approach.

Index Terms—Large language models, generative artificial intelligence, cloud manufacturing, chain-of-thought, Internet of Things

I. INTRODUCTION

As large language models (LLMs) are progressing rapidly, the adoption of LLMs in manufacturing applications is continuously increasing. This is especially true for scheduling problems that require semantic understanding and multi-step decision support. For example, the authors in [1] developed an LLM-based multi-agent scheduling chain for the flexible job shop scheduling problem. In this framework, an improved decision-making process is used to analyze scheduling indicators and select suitable algorithms. The overall framework also supports dynamic scheduling and real-time rescheduling. More recently, [2] proposed an LLM-empowered dynamic scheduling framework for intelligent hybrid flow shops. In this framework, structured prompt engineering is used to enrich state representation and guide action selection within a multi-agent deep reinforcement learning (DRL) framework. LLMs have also been utilized to improve the performance of solving combinatorial optimization problems [3]. LLMs have been applied to tackle the job shop scheduling problem [4], and integrated into a multi-agent manufacturing system to handle dynamic changes in production tasks [5].

Moreover, cloud manufacturing (CMfg), a key paradigm of smart manufacturing, enables service-oriented sharing and efficient allocation of geographically dispersed resources, where coordinated decision-making across cyber-physical components is essential to achieve its value [6]. With the continued advancement of Internet of Things (IoT) technologies, CMfg increasingly relies on cloud-edge-device collaboration, in which computational tasks (e.g., digital twins, real-time analytics, and predictive maintenance) play a critical role in supporting time-sensitive industrial operations [7]. In a typical CMfg setting, physical logistics operations can be executed by automated agents (e.g., unmanned aerial vehicles (UAVs) [8] or automated guided vehicles [9]), while computational workloads are processed at the device, edge, or cloud layers [10]. These computational workloads are typically generated by industrial

This work was supported in part by the Agency for Science, Technology and Research (A*STAR) under its Manufacturing, Trade & Connectivity (MTC) Individual Research Grants (IRG) under Grant M23M6c0113; in part by MTC Programmatic under Grant M23L9b0052; in part by the Seatrium New Energy Laboratory, Singapore Ministry of Education (MOE) Tier 1, under Grant RT5/23 and Grant RG24/24; in part by the Nanyang Technological University (NTU) Center for Computational Technologies in Finance (NTU-CCTF); and in part by the Research Innovation and Enterprise (RIE) 2025 Industry Alignment Fund-Industry Collaboration Projects (IAF-ICP), administered by A*STAR, under Grant I2301E0026. (Corresponding author: Wei Zhang.)

Hanwen Zhang is with both the College of Computing and Data Science, Nanyang Technological University, Singapore 639798, and the Information and Communications Technology Cluster, Singapore Institute of Technology, Singapore 828608 (e-mail: hanwen001@e.ntu.edu.sg and hanwen.zhang@singaporetech.edu.sg)

Dusit Niyato is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dniyato@ntu.edu.sg).

Wei Zhang, Xin Lou, and Malcolm Yoke Hean Low are with the Information and Communications Technology Cluster, Singapore Institute of Technology, Singapore 828608 (e-mail: wei.zhang@singaporetech.edu.sg, lou.xin@singaporetech.edu.sg, and malcolm.low@singaporetech.edu.sg).

sensing and monitoring applications, such as image-recognition-based quality inspection, anomaly detection, real-time analytics, and predictive maintenance. In this paper, we study an integrated scheduling problem that jointly optimizes UAV-assisted product collection and cross-layer computational task offloading from devices to UAVs and the cloud. This joint scheduling problem is a challenging combinatorial optimization task due to the tight coupling between UAV routing/service availability and task execution decisions under stringent resource and deadline constraints, as discussed below.

Challenge I: The mathematical modeling of combinatorial optimization problems in CMfg can be significantly more complex than that of traditional single-domain scheduling problems. Similar coupling complexity has also been observed in UAV-assisted MEC systems, where computational task offloading is jointly optimized with UAV trajectory, path-planning, or task-scheduling decisions [11]. It requires capturing how UAV routing induces time-varying service windows. These service windows, in turn, affect per-slot task offloading and compute/communication allocation under energy budgets, capacity limits, and task deadlines. At the same time, the model must balance conflicting goals, such as logistics efficiency and sufficient resource allocation for meeting task deadlines. These complexities lead to a complicated modeling process. As a result, the problem becomes challenging for engineers and researchers to model, formulate, and solve such complex problems. This issue is due to the fact that specific operational dynamics of joint logistics-computation scheduling CMfg have to be taken into account. **Challenge II:** Compared with traditional scheduling problems, hybrid task scheduling in CMfg introduces significantly higher solution complexity. The reason is that logistics- and computation-related decisions must be optimized jointly rather than handled in isolation. Moreover, as user demands and system conditions evolve over time, the resulting optimization problem may also change. Therefore, the key challenge is to solve these changing optimization problems efficiently. In particular, the scheduler must jointly determine UAV routing, offloading-destination selection, and compute/communication allocation under strict latency, resource, and communication constraints. Therefore, designing DRL frameworks that can generalize across different hybrid task scenarios while mitigating sparse feedback and complex action representation is crucial for enhancing the efficiency and adaptability of intelligent CMfg systems.

To formulate complex combinatorial problems in CMfg, we present an agentic AI framework to address **Challenge I**. The agent facilitates dialogue-driven mathematical model generation and refinement by combining LLM reasoning with retrieval augmented generation (RAG) and CoT reasoning. In particular, an LLM converts natural language descriptions into structured modeling logic. Then, RAG extracts pertinent formulations, constraints, and objectives from curated literature and technical documentation. CoT further makes explicit the logic underlying variable definitions, constraint design, and objective construction. Hence, CoT can enhance the accuracy and transparency of the model. In CMfg settings where decisions couple UAV routing, offloading-destination selection, and compute/communication resource allocation, the proposed

agentic AI helps UAV operators efficiently derive interpretable, domain-specific formulations.

To address **Challenge II**, we propose a hierarchical DRL approach to solve the coupled UAV-assisted collection and mobile edge computing (MEC) scheduling problem. Inspired by [8], the framework decomposes the original joint optimization into two tractable sequential decision layers, which substantially reduces the effective state/action space while preserving the essential coupling between logistics and computation services. Both layers are formulated as finite-horizon Markov decision processes (MDPs). They are solved using proximal policy optimization (PPO). This helps maintain stable policy updates under high-dimensional and constrained decision making. Specifically, the *upper layer* (UAV routing) formulates the multi-UAV routing problem as a single-agent MDP. It jointly determines the manufacturing-station assignment for each UAV and the corresponding visiting order under energy, payload, and flight-distance constraints. Given the resulting routes, the *lower layer* (computational task assignment) makes per-slot execution decisions—offloading destination (local/UAV/cloud via UAV) and compute/communication allocation—under capacity and deadline constraints. The two layers interact through two signals. The upper layer provides service windows and remaining UAV energy to the lower layer, which uses them to determine the feasibility of UAV-involved processing.

This is the first work in the field of smart manufacturing that utilizes agentic AI, i.e., LLMs with RAG and CoT reasoning, to formulate hybrid combinatorial optimization problems in CMfg. The formulated problems are then solved using a hierarchical DRL approach. The contributions of this work are summarized as follows:

- We propose an agentic AI framework that integrates LLMs, RAG, and CoT reasoning to assist in formulating hybrid combinatorial problems in CMfg. This framework enables the decomposition of complex task requirements into structured optimization models, capturing the intricate dependencies among logistics and computational tasks. (For **Challenge I**)
- We design an interactive problem formulation process that leverages RAG to retrieve domain-specific knowledge and CoT to enhance interpretability and traceability of the generated models. This process allows the agentic AI to reason step-by-step about task couplings, performance requirements, resource constraints, and scheduling objectives. Hence, the process enables accurate and adaptive model construction under cloud-edge-device collaboration. (For **Challenge I**)
- We develop a hierarchical DRL approach to solve the formulated hybrid scheduling problem. By decomposing the coupled routing-MEC decision making into an upper routing layer and a lower task-scheduling layer, the framework improves training stability while preserving cross-layer coordination through routing-induced service availability. (For **Challenge II**)

The structure of this paper is outlined as follows. In Section II, we provide an overview of the relevant literature. The proposed generative AI agent framework is introduced in Section V. We discuss the system model in Section III.

In Section VI, we present the hierarchical DRL approach. Simulations of the generative AI agent framework and the hierarchical DRL approach are discussed in Section VII. Finally, Section VIII offers a summary and conclusions of the study.

II. RELATED WORK

This section comprehensively reviews prior studies in three related areas, namely the UAVs routing problem, computational task offloading problem, and agentic AI. It discusses the progress achieved in these domains while outlining the limitations that motivate and position the contributions of this work.

A. UAVs Routing Problem

UAV routing has attracted growing attention in a range of applications, including aerial survey, logistics delivery, and maritime services. For example, the authors in [12] studied a glider routing problem for aerial survey and jointly considered routing and flight trajectory optimization through a matheuristic framework. The authors in [13] formulated a dynamic drone routing problem with uncertain demand and energy consumption as an MDP and developed an approximate dynamic solution for routing, drone usage, and battery management. From a learning-based perspective, the authors in [14] proposed a multi-agent DRL framework for dynamic logistics UAV routing in large-scale multi-UAV systems. In maritime settings, a collaborative vessel-UAV routing problem with time windows for offshore delivery was discussed in [15]. Moreover, a shore-to-ship drone routing problem with non-stationary vessel locations and non-linear energy consumption was presented in [16]. Despite these advances, existing studies are often built on specialized assumptions and complex formulations, which can make their modeling logic difficult to follow. This motivates the need for a clearer and more accessible framework for UAV routing problems in coupled cyber-physical systems.

B. Computational Task Offloading Problem

Computational task offloading has become a key topic in edge intelligence because efficient offloading is essential for balancing latency, energy consumption, and resource utilization in dynamic computing systems. Existing studies have explored this problem from several perspectives. For example, the authors in [17] studied mobility-aware dependent task offloading in collaborative edge computing and proposed a digital twin-assisted DRL method for dynamic offloading and bandwidth-related decision making. The authors in [18] investigated delay-sensitive task offloading with edge caching and developed a soft actor-critic based scheme to jointly optimize offloading, caching, and computation/resource allocation under delay reliability constraints. Recent works have also considered collaborative and hierarchical settings. The authors in [19] proposed a digital twin-assisted multi-agent reinforcement learning framework for reliability-aware dependent task offloading in decentralized edge computing, jointly considering bandwidth allocation. In [20], a hierarchical aerial computing framework that combined UAVs and a high-altitude platform was used

to study the joint offloading, user association, and resource allocation. The authors in [21] explored the joint task offloading and resource allocation in ultra-dense MEC using a mean-field-based DRL method to make the system more scalable. Although these studies have advanced task offloading by considering more complex architectures and adaptive learning-based methods, their formulations are often highly specialized and difficult to follow. This motivates the need for a clearer and more accessible framework for computational task offloading in cloud-edge-device systems.

C. Agentic AI

Recent advances in agentic AI have extended LLM-based systems from question answering to interactive agents with reasoning, planning, retrieval, and tool-use capabilities [22]. In manufacturing, prior studies have explored this trend from several perspectives. For example, the authors in [23] reviewed the concepts, capability boundaries, and challenges of agentic AI for future manufacturing. The authors in [24] further proposed a manufacturing-oriented agentic AI framework with coordinated LLM-based agents, a unified data-model-knowledge lake, and human-in-the-loop oversight. Another line of research focuses on domain-grounded interaction through retrieval augmentation. In [25], the authors developed a RAG-based LLM for industrial knowledge management over heterogeneous enterprise documents. The authors in [26] proposed an agentic-AI-assisted framework for advanced planning and scheduling with multi-agent coordination and RAG-CoT-enhanced interaction. In [27], the authors studied agentic data analysis in manufacturing through a lightweight workflow for natural-language interaction with manufacturing datasets. Although these studies highlight the value of agentic AI in manufacturing, they mainly focus on conceptual frameworks, industrial question answering, scheduling assistance, or data interaction. Its use for customized mathematical system modeling remains limited, especially for selectively retrieving domain knowledge and organizing it into an interpretable optimization formulation for tightly coupled hybrid problems. This gap motivates the framework proposed in this work.

III. SYSTEM MODEL

In this section, we provide a system overview, a UAV routing model, and a MEC service model. The relevant symbols are summarized in Tables I and II.

A. System Overview

A typical manufacturing sequence consists of material delivery, manufacturing completion, and product collection. Accordingly, the logistics process has two phases, namely phase 1 for material delivery and phase 2 for product collection. In this paper, we focus on phase 2. Specifically, a fleet of U homogeneous UAVs is dispatched from a central depot equipped with terrestrial MEC servers, hereafter referred to as the cloud, to collect finished products from M geographically distributed manufacturing stations. At the same time, these UAVs provide MEC services to industrial sensor devices (ISDs)

TABLE I
SYMBOLS: SUPERSCRIPTS AND NOTATIONS

Symbols	Description
Superscripts	
cmp, com	Computing-related / communication-related.
fly, hov	Flight / hovering energy tag.
loc, uav, cld	Local / UAV / cloud mode or capacity type.
ul, bh	Uplink (ISD→UAV) / backhaul (UAV→cloud).
max	Budget upper bound.
Other Notations	
$\chi_{k,\tau}(t)$	Cumulative completion flag by slot t .
$E_u^{\text{cmp}}, E_u^{\text{com}}$	UAV onboard-computing / communication energy.
E_u^{fly}	UAV flight-hover energy.
$\mathcal{I}$	Realized-task set indexed by (k, τ) .
L	Set of routing nodes, $L = \{0\} \cup S$, 0 denotes depot.
$m(k)$	Station hosting ISD k .
m, u, k, i, j	Indices: station / UAV / ISD / node i / node j .
N_{slot}	Total number of time slots in $\mathcal{T} = \{0, \dots, N_{\text{slot}}-1\}$.
R_t^{cmp}	Normalized computing occupation in slot t .
R_t^{com}	Normalized communication occupation in slot t .
S, U, K	Sets of stations / UAVs / ISDs.
$\mathcal{T}$	Time slot set; $t \in \mathcal{T}$.
τ, t	Indices: task-generation slot / slot.

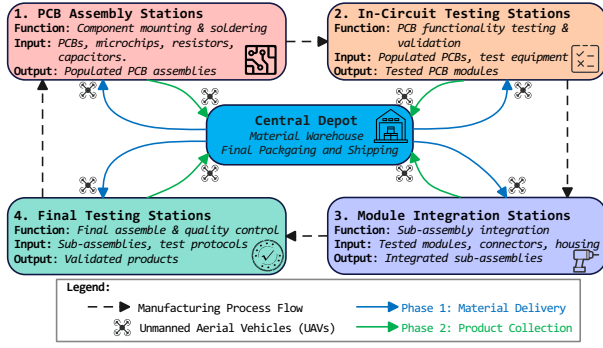


Fig. 1. An example of UAV-assisted electronics manufacturing: Two-phase logistic coordination system. The manufacturing process flow includes PCB assembly, in-circuit testing, module integration, and final testing. The UAVs handle logistics in two phases: delivering materials before manufacturing in phase 1 and collecting products after manufacturing in phase 2.

deployed at the stations. All UAVs depart from and return to the depot, and they fly at a fixed altitude with constant speed. Each station $m \in S$ is located at (X_m, Y_m) , has a collection reward v_m , product weight w_m , and hosts a set of ISDs K_m . All finished products are assumed to be ready for collection at the mission start, while ISDs generate computational tasks stochastically over a discretized time horizon.

The considered system couples physical logistics with computational task processing. On the logistics side¹, each UAV is constrained by payload, travel distance, mission time, and battery energy, and must complete a depot-based collection mission [28]. On the computing side, task execution follows a three-layer cloud-edge-device architecture, where a task can be processed locally at the ISD, at a serving UAV, or at the cloud via a UAV-assisted path [10]. Since UAV-assisted processing is available only when a UAV is serving the corresponding station,

¹<https://wing.com/>

TABLE II
SYMBOLS: DECISION VARIABLES AND PARAMETERS

Symbols	Description
Decision Variables	
$b_{k,\tau,t,u}^{\text{bh}}$	UAV- $u \rightarrow$ cloud rate allocated (bits/s).
$b_{k,\tau,t,u}^{\text{ul}}$	ISD→UAV- u rate allocated (bits/s).
c_m	Binary: station m collected.
$\delta_{k,\tau,t}$	Binary: task (k, τ) completes in slot t .
$\eta_{u,m,t}$	Binary: UAV u serves station m in slot t .
$f_{k,\tau,t,u}^{\text{cld}}$	Cloud compute via UAV u allocated (work-unit/s).
$f_{k,\tau,t,u}^{\text{uav}}$	UAV- u compute allocated (work-unit/s).
$f_{k,\tau,t}^{\text{loc}}$	Local compute allocated (work-unit/s).
$g_{k,\tau,u}$	Binary: task (k, τ) executed in cloud via UAV u .
$p_{k,\tau,u}$	Binary: task (k, τ) processed on UAV u .
$\zeta_{k,\tau}$	Binary: task (k, τ) processed locally.
$l_{k,\tau,u}^{\text{th}}$	Binary: UAV u is used as the first hop for task (k, τ) .
s_u, r_u	Depot departure / return time of UAV u .
$T_{k,\tau}$	Task completion time.
$T_{u,m}^{\text{arr}}, T_{u,m}^{\text{dep}}$	Arrival / departure time of UAV u at station m .
$x_{u,i,j}$	Binary: UAV u travels $i \rightarrow j$.
$y_{u,m}$	Binary: station m assigned to UAV u .
$z_{k,\tau}$	Binary: task (k, τ) meets deadline.
Parameters	
α_{bh}	Energy coefficient for UAV-to-cloud communication.
α_{cmp}	Energy coefficient for onboard computing.
$\alpha_{\text{fly}}, \alpha_{\text{hov}}$	Energy coefficients for flying and hovering.
α_{ul}	Energy coefficient for ISD-to-UAV communication.
$B_{k,\tau}$	Task input size (bits).
$B_u^{\text{ul}}, B_u^{\text{bh}}$	UAV u uplink / backhaul limits (bits/s).
$\gamma_{k,t,u}^{\text{ul}}, \gamma_{t,u}^{\text{bh}}$	Effective-rate factors (uplink / backhaul).
Δ	Time slot duration.
$d_{i,j}$	Distance between nodes i and j .
D_k	Task deadline duration for ISD k tasks.
D_u	UAV distance budget.
E_u^{max}	UAV u energy budget.
F^{cld}	Cloud compute capacity (work-unit/s).
F_k^{loc}	ISD k local compute capacity (work-unit/s).
F_u^{uav}	UAV u compute capacity (work-unit/s).
$\omega_{\text{col}}, \omega_{\text{cmp}}$	Weights for collection value and task completion.
$\omega_{\text{miss}}, \omega_{\text{flow}}$	Weights for miss penalty and flow-time.
ω_{res}	Weight for resource-occupation cost.
Q_u	UAV payload capacity.
τ_m	Minimum service time at station m .
T_{mission}	Mission horizon.
v_{fly}	UAV flight speed.
v_m, w_m	Product value / weight at station m .
$W_{k,\tau}$	Task workload (work-units).

routing decisions directly determine the service windows for MEC tasks [11]. Therefore, the system is modeled from two coupled perspectives in the following subsections: the *UAV Routing Model*, which captures UAV collection decisions and routing feasibility, and the *MEC Service Model*, which captures task execution and resource-allocation decisions under the routing-induced service availability.

A representative application of the proposed system model is the electronics component assembly and testing industry, where geographically distributed stations perform operations such as PCB assembly, in-circuit testing, module integration, and final testing. As illustrated in Fig. 1, UAVs support two-phase logistics: They first deliver raw materials or sub-

assembled components to the stations and then collect finished products for transporting to a central depot after processing is completed [29]. During both phases, the UAVs can also provide MEC support by executing offloaded tasks on board or relaying them to the cloud, thereby supporting timely processing of industrial sensing and analytics tasks. Such UAV-assisted operation is well suited to modern manufacturing environments, where UAVs are increasingly used for delivery, monitoring, inspection, inventory management, and predictive maintenance [30]. Compared with ground vehicles, UAVs offer three-dimensional mobility, can access elevated stations, bypass floor obstacles, and operate without dedicated ground infrastructure, making them effective in spatially constrained and complex industrial settings [31], [32].

B. UAV Routing Model

The UAV-routing constraints are organized into three parts, namely collection assignment and route consistency, payload and distance budgets, and mission timing feasibility.

1) Collection assignment and route consistency:

$$\sum_{u \in U} y_{u,m} \leq 1, \quad c_m \leq \sum_{u \in U} y_{u,m}, \quad \forall m \in S, \quad (1)$$

$$\sum_{i \in L} x_{u,i,m} = \sum_{j \in L} x_{u,m,j} = y_{u,m}, \quad \forall u \in U, \quad \forall m \in S, \quad (2)$$

$$\sum_{m \in S} x_{u,0,m} = \sum_{m \in S} x_{u,m,0}, \quad \forall u \in U. \quad (3)$$

Eq. (1) limits each station to at most one UAV and allows a station to be counted as collected only if it is assigned. Eq. (2) ensures that station m appears on the route of UAV u exactly when it is assigned to UAV u , in which case it is entered and left exactly once by UAV u . Eq. (3) ensures that each UAV departs from and returns to the depot the same number of times.

2) Payload and distance budgets:

$$\sum_{m \in S} w_m y_{u,m} \leq Q_u, \quad \sum_{i \in L} \sum_{j \in L} d_{i,j} x_{u,i,j} \leq D_u, \quad \forall u \in U. \quad (4)$$

Eq. (4) limits the total assigned payload weight and the total travel distance of each UAV.

3) Mission timing:

$$s_u = 0, \quad r_u \leq T_{\text{mission}}, \quad \forall u \in U, \quad (5)$$

$$x_{u,0,m} = 1 \Rightarrow T_{u,m}^{\text{arr}} \geq s_u + \frac{d_{0,m}}{v_{\text{fly}}}, \quad \forall u \in U, \quad \forall m \in S, \quad (6)$$

$$x_{u,i,j} = 1 \Rightarrow T_{u,j}^{\text{arr}} \geq T_{u,i}^{\text{dep}} + \frac{d_{i,j}}{v_{\text{fly}}}, \quad \forall u \in U, \quad \forall i, j \in S, \quad (7)$$

$$x_{u,m,0} = 1 \Rightarrow r_u \geq T_{u,m}^{\text{dep}} + \frac{d_{m,0}}{v_{\text{fly}}}, \quad \forall u \in U, \quad \forall m \in S, \quad (8)$$

$$T_{u,m}^{\text{dep}} - T_{u,m}^{\text{arr}} \geq \tau_m y_{u,m}, \quad \forall u \in U, \quad \forall m \in S. \quad (9)$$

Eq. (5) states that each UAV starts at time 0 and must return within the mission horizon. Eqs. (6) to (8) ensure that the route timing is consistent with the corresponding travel times. In Eq. (7), the case $i = j$ is excluded, since no travel is needed from a station to itself. Eq. (9) requires a minimum service duration at each station assigned to UAV u .

C. MEC Service Model

The constraints associated with MEC service model are about task offloading, including execution-mode selection, completion-time encoding, computing and communication resource limits, service-window indicators, service-window feasibility for UAV-involved processing, deadline satisfaction, and UAV energy budgeting.

1) Execution-mode selection:

$$\zeta_{k,\tau} + \sum_{u \in U} p_{k,\tau,u} + \sum_{u \in U} g_{k,\tau,u} = 1, \quad \forall (k, \tau) \in \mathcal{I}. \quad (10)$$

Eq. (10) requires every task to choose exactly one execution mode: local execution, UAV execution, or cloud execution via a UAV relay.

2) Completion-time encoding:

$$z_{k,\tau} = \sum_{t=\tau}^{N_{\text{slot}}-1} \delta_{k,\tau,t}, \quad \forall (k, \tau) \in \mathcal{I}, \quad (11)$$

$$T_{k,\tau} = \sum_{t=\tau}^{N_{\text{slot}}-1} (t+1) \Delta \delta_{k,\tau,t} + T_{\text{mission}}(1 - z_{k,\tau}). \quad (12)$$

Eqs. (11) and (12) define the completion-status indicator $z_{k,\tau}$ and the completion time $T_{k,\tau}$ based on the slot-selection binary $\delta_{k,\tau,t} \in \{0, 1\}$. Specifically, $\delta_{k,\tau,t} = 1$ indicates that task (k, τ) is completed by the end of slot t .

3) Computing and communication resource constraints:

The computing and communication resources are subject to three types of constraints, namely per-slot capacity limits, mode consistency, and cumulative service sufficiency.

(i) Per-slot capacity limits

$$\begin{cases} \sum_{\tau': (k, \tau') \in \mathcal{I}, \tau' \leq t} f_{k,\tau',t}^{\text{loc}} \leq F_k^{\text{loc}}, & \forall k \in K, \quad \forall t \in \mathcal{T}, \\ \sum_{(k, \tau) \in \mathcal{I}: \tau \leq t} f_{k,\tau,t,u}^{\text{uav}} \leq F_u^{\text{uav}}, & \forall u \in U, \quad \forall t \in \mathcal{T}, \\ \sum_{u \in U} \sum_{(k, \tau) \in \mathcal{I}: \tau \leq t} f_{k,\tau,t,u}^{\text{cld}} \leq F^{\text{cld}}, & \forall t \in \mathcal{T}. \end{cases} \quad (13)$$

$$\begin{cases} \sum_{(k, \tau) \in \mathcal{I}: \tau \leq t} b_{k,\tau,t,u}^{\text{ul}} \leq B_u^{\text{ul}}, & \forall u \in U, \quad \forall t \in \mathcal{T}, \\ \sum_{(k, \tau) \in \mathcal{I}: \tau \leq t} b_{k,\tau,t,u}^{\text{bh}} \leq B_u^{\text{bh}}, & \forall u \in U, \quad \forall t \in \mathcal{T}. \end{cases} \quad (14)$$

Eqs. (13) and (14) bound the aggregate computing and communication resource allocations in each slot by the available local, UAV, cloud, uplink, and backhaul capacities.

(ii) Mode Consistency

$$\begin{cases} 0 \leq f_{k,\tau,t}^{\text{loc}} \leq F_k^{\text{loc}} \zeta_{k,\tau}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall t \geq \tau, \\ 0 \leq f_{k,\tau,t,u}^{\text{uav}} \leq F_u^{\text{uav}} p_{k,\tau,u}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall u \in U, \quad \forall t \geq \tau, \\ 0 \leq f_{k,\tau,t,u}^{\text{cld}} \leq F^{\text{cld}} g_{k,\tau,u}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall u \in U, \quad \forall t \geq \tau, \end{cases} \quad (15)$$

$$\begin{cases} 0 \leq b_{k,\tau,t,u}^{\text{ul}} \leq B_u^{\text{ul}} \iota_{k,\tau,u}^{\text{fh}}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall u \in U, \quad \forall t \geq \tau, \\ 0 \leq b_{k,\tau,t,u}^{\text{bh}} \leq B_u^{\text{bh}} g_{k,\tau,u}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall u \in U, \quad \forall t \geq \tau, \\ \iota_{k,\tau,u}^{\text{fh}} = p_{k,\tau,u} + g_{k,\tau,u}. \end{cases} \quad (16)$$

Eqs. (15) and (16) impose mode consistency on computing and communication allocations by requiring each allocation to be zero unless the corresponding execution mode is selected.

(iii) Multi-slot sufficiency for completion Define $\chi_{k,\tau}(t) \triangleq \sum_{r=\tau}^t \delta_{k,\tau,r} \in \{0, 1\}$, which indicates whether task (k, τ) has been completed by the end of slot t . Then, for any $t \geq \tau$, the cumulative service must be sufficient whenever completion by slot t is claimed. Then, we have the following constraints:

$$\begin{cases} \zeta_{k,\tau} = 1 \Rightarrow \sum_{s=\tau}^t f_{k,\tau,s}^{\text{loc}} \Delta \geq W_{k,\tau} \chi_{k,\tau}(t), \\ p_{k,\tau,u} = 1 \Rightarrow \sum_{s=\tau}^t f_{k,\tau,s,u}^{\text{uav}} \Delta \geq W_{k,\tau} \chi_{k,\tau}(t), \\ g_{k,\tau,u} = 1 \Rightarrow \sum_{s=\tau}^t f_{k,\tau,s,u}^{\text{cld}} \Delta \geq W_{k,\tau} \chi_{k,\tau}(t), \\ \text{where, } \forall (k, \tau) \in \mathcal{I}, \quad \forall t \geq \tau, \quad \forall u \in U. \end{cases} \quad (17)$$

$$\begin{cases} l_{k,\tau,u}^{\text{fh}} = 1 \Rightarrow \sum_{s=\tau}^t \gamma_{k,s,u}^{\text{ul}} b_{k,\tau,s,u}^{\text{ul}} \Delta \geq B_{k,\tau} \chi_{k,\tau}(t), \\ g_{k,\tau,u} = 1 \Rightarrow \sum_{s=\tau}^t \gamma_{s,u}^{\text{bh}} b_{k,\tau,s,u}^{\text{bh}} \Delta \geq B_{k,\tau} \chi_{k,\tau}(t), \\ \text{where, } \forall (k, \tau) \in \mathcal{I}, \quad \forall t \geq \tau, \quad \forall u \in U. \end{cases} \quad (18)$$

Eq. (17) ensures that a task can be declared completed only after receiving enough cumulative computing service. Eq. (18) imposes the same idea on communication: the cumulative transmitted bits must be no smaller than the task input size whenever completion is claimed.

4) *Service-window indicators:*

$$\sum_{m \in S} \eta_{u,m,t} \leq 1, \quad \forall u \in U, \quad \forall t \in \mathcal{T}, \quad (19)$$

$$\eta_{u,m,t} \leq y_{u,m}, \quad \forall u \in U, \quad \forall m \in S, \quad \forall t \in \mathcal{T}, \quad (20)$$

$$\eta_{u,m,t} = 1 \Rightarrow T_{u,m}^{\text{arr}} \leq t \Delta \leq T_{u,m}^{\text{dep}}, \quad (21)$$

where, $\forall u \in U, \quad \forall m \in S, \quad \forall t \in \mathcal{T}$.

Eqs. (19) to (21) define the slot-level service-window indicators by allowing service only at assigned stations and within the corresponding arrival-departure interval.

5) *Service-window feasibility for UAV-involved processing:*

Let $m(k)$ denote the station to which ISD k belongs. Then,

$$\begin{aligned} b_{k,\tau,t,u}^{\text{ul}} &\leq B_u^{\text{ul}} \eta_{u,m(k),t}, & f_{k,\tau,t,u}^{\text{uav}} &\leq F_u^{\text{uav}} \eta_{u,m(k),t}, \\ b_{k,\tau,t,u}^{\text{bh}} &\leq B_u^{\text{bh}} \eta_{u,m(k),t}, & \forall (k, \tau) \in \mathcal{I}, \quad \forall u \in U, \quad \forall t \geq \tau. \end{aligned} \quad (22)$$

Eq. (22) requires the UAV-involved communication and computing allocations of task (k, τ) in slot t to be zero unless UAV u is serving station $m(k)$ in that slot.

6) *Deadline constraint:*

$$z_{k,\tau} = 1 \Rightarrow T_{k,\tau} \leq \tau \Delta + D_k, \quad \forall (k, \tau) \in \mathcal{I}. \quad (23)$$

Eq. (23) states that a task is marked as on-time only if its completion time is no later than its deadline.

7) *UAV energy budget:* The total energy of UAV u is decomposed into flight/hover energy, onboard computing energy, and communication energy:

$$\begin{aligned} E_u^{\text{fly}} &= \alpha_{\text{fly}} \sum_{i \in L} \sum_{j \in L} d_{i,j} x_{u,i,j} \\ &\quad + \alpha_{\text{hov}} \sum_{m \in S} (T_{u,m}^{\text{dep}} - T_{u,m}^{\text{arr}}) y_{u,m}, \end{aligned} \quad (24)$$

$$E_u^{\text{cmp}} = \alpha_{\text{cmp}} \sum_{t \in \mathcal{T}} \sum_{(k,\tau) \in \mathcal{I}: \tau \leq t} f_{k,\tau,t,u}^{\text{uav}} \Delta, \quad (25)$$

$$\begin{aligned} E_u^{\text{com}} &= \alpha_{\text{ul}} \sum_{t \in \mathcal{T}} \sum_{(k,\tau) \in \mathcal{I}: \tau \leq t} \gamma_{k,t,u}^{\text{ul}} b_{k,\tau,t,u}^{\text{ul}} \Delta \\ &\quad + \alpha_{\text{bh}} \sum_{t \in \mathcal{T}} \sum_{(k,\tau) \in \mathcal{I}: \tau \leq t} \gamma_{t,u}^{\text{bh}} b_{k,\tau,t,u}^{\text{bh}} \Delta. \end{aligned} \quad (26)$$

The total energy must satisfy

$$E_u^{\text{fly}} + E_u^{\text{cmp}} + E_u^{\text{com}} \leq E_u^{\text{max}}, \quad \forall u \in U. \quad (27)$$

Eqs. (24) to (26) define the flight/hover, onboard computing, and communication energy components of UAV u , and Eq. (27) imposes the corresponding per-UAV energy budget over the whole mission.

IV. PROBLEM FORMULATION AND ANALYSIS

In this section, we provide a problem formulation and a problem analysis.

A. Problem Formulation

We aim to maximize the overall system performance, subject to the UAV routing constraints in Eq. (1)–Eq. (9) and the MEC service (i.e., task offloading) constraints in Eq. (10)–Eq. (27). Specifically, the objective promotes product collection and deadline-compliant task completion, while reducing deadline violations, task completion delay, and overall computing and communication resource occupation. The complete optimization problem is formulated as follows:

$$\begin{aligned} \mathbf{P}: \quad \max \quad & \underbrace{\omega_{\text{col}} \sum_{m \in S} v_m c_m}_{\text{collection value}} + \underbrace{\omega_{\text{cmp}} \sum_{(k,\tau) \in \mathcal{I}} z_{k,\tau}}_{\text{on-time completion return}} \\ & - \underbrace{\omega_{\text{miss}} \sum_{(k,\tau) \in \mathcal{I}} (1 - z_{k,\tau})}_{\text{deadline violation cost}} - \underbrace{\omega_{\text{flow}} \sum_{(k,\tau) \in \mathcal{I}} (T_{k,\tau} - \tau \Delta)}_{\text{flow-time cost}} \\ & - \underbrace{\omega_{\text{res}} \sum_{t \in \mathcal{T}} (R_t^{\text{cmp}} + R_t^{\text{com}})}_{\text{resource-occupation cost}} \end{aligned} \quad (28)$$

s.t. Eq. (1) – Eq. (9),

Eq. (10) – Eq. (27).

where $T_{k,\tau}$ denotes the completion time of task (k, τ) , $z_{k,\tau} \in \{0, 1\}$ indicates whether the task is completed before its deadline, and R_t^{cmp} and R_t^{com} denote the normalized computing and communication occupation in slot t , respectively.

B. Problem Analysis

The mathematical formulation presented in Section IV-A is a large-scale mixed-integer optimization problem that jointly couples multi-UAV routing and slot-level task offloading. In terms of computational complexity, the UAV routing part is NP-hard since it jointly determines station assignment and visit sequencing under route-consistency, payload, distance, and mission time constraints while maximizing collection value, which amounts to a capacitated value-collecting multi-UAV routing variant. The task offloading part is also NP-hard. To see this, consider a restricted case where all routing-related quantities are fixed, service windows are predetermined, there is only one time slot, communication and energy budgets are set sufficiently large so that they do not constrain the scheduling decisions, and only UAV execution is allowed. This restricted case still subsumes the classical PARTITION problem. PARTITION refers to the problem of deciding whether a set of positive integers can be divided into two subsets with equal sums. Therefore, both subproblems, and thus the full joint formulation, are NP-hard, which motivates our proposed hierarchical DRL approach in Section VI as a scalable solution method.

V. PROPOSED AGENTIC AI FRAMEWORK: LLM WITH CHAIN-OF-THOUGHT

In this section, we introduce the proposed agentic AI framework shown in Fig. 2, where the user submits a natural-language modeling request, the request is grounded by the RAG module through retrieval of relevant contextual knowledge, the *Agentic AI Responder* generates the formulation through CoT reasoning, and the *Agentic AI Verifier* validates each reasoning step before the process proceeds. We discuss the retrieval-augmented generation (RAG) processes, and the CoT-based generation-and-verification workflow.

A. Retrieval-Augmented Generation

With the rapid development of LLMs, RAG has become a promising approach for improving the factual grounding and domain consistency of LLM outputs [33], [34]. The RAG module [35] is used to ground the LLM with task-relevant evidence before model generation. Since no public corpus directly matches the hybrid UAV-assisted collection and MEC scheduling problem considered in this paper, we organize an internal knowledge base from two complementary perspectives: UAV routing and computational task offloading. The repository contains modeling-oriented knowledge, including system descriptions, notation references, variable definitions, objective patterns, constraint templates, and representative formulation fragments in Section IX-A. The retrieved knowledge is organized as text, segmented into chunks, and indexed for semantic retrieval [25].

Let ξ denote a user request, and let $\mathcal{H} = \{h_1, h_2, \dots, h_N\}$ be the chunked knowledge repository. A shared text encoder $\Psi_{\text{enc}}(\cdot)$ maps both the query ξ and each chunk h_n into the same embedding space [36], which we write as:

$$\mathbf{r}_\xi = \Psi_{\text{enc}}(\xi), \quad \mathbf{r}_n = \Psi_{\text{enc}}(h_n), \quad n = 1, \dots, N, \quad (29)$$

where $\mathbf{r}_\xi, \mathbf{r}_n \in \mathbb{R}^{d_{\text{emb}}}$ are dense semantic vectors and d_{emb} is the embedding dimension. To measure relevance between the query and each chunk, we use cosine similarity [37] defined as follows:

$$\text{sim}(\xi, h_n) = \frac{\mathbf{r}_\xi^\top \mathbf{r}_n}{\|\mathbf{r}_\xi\|_2 \|\mathbf{r}_n\|_2}, \quad n = 1, \dots, N. \quad (30)$$

A larger $\text{sim}(\xi, h_n)$ indicates that chunk h_n is more semantically aligned with the query. Given a retrieval budget K_{ret} , we define the index set of the top- K_{ret} most relevant chunks as [38]

$$\mathcal{J}_{K_{\text{ret}}}(\xi) = \text{Top}_{K_{\text{ret}}} \{ \text{sim}(\xi, h_n) \}_{n=1}^N, \quad (31)$$

where $\mathcal{J}_{K_{\text{ret}}}(\xi)$ denotes the set of indices of the retrieved chunks for query ξ , and $\text{Top}(\cdot)$ returns the indices corresponding to the K_{ret} largest values in the similarity-score collection $\{ \text{sim}(\xi, h_n) \}_{n=1}^N$. The retrieved chunks form the context used in the subsequent generation stage [39], which we denote by

$$\mathcal{R}_{\text{rag}}(\xi) = \{ h_n \mid n \in \mathcal{J}_{K_{\text{ret}}}(\xi) \}, \quad (32)$$

where $\mathcal{R}_{\text{rag}}(\xi)$ denotes the set of retrieved text chunks for query ξ , and each chunk h_n is included if its index n belongs to the top- K_{ret} index set $\mathcal{J}_{K_{\text{ret}}}(\xi)$.

The retrieved chunks are then combined with the original query through an aggregation operator to form the final prompt [40]:

$$\mathbf{p}(\xi) = \text{Agg}(\xi, \mathcal{R}_{\text{rag}}(\xi)), \quad (33)$$

where $\text{Agg}(\cdot, \cdot)$ denotes the prompt-construction operator, and $\mathbf{p}(\xi)$ is the retrieval-augmented prompt formed by combining the user request ξ with the retrieved chunk set $\mathcal{R}_{\text{rag}}(\xi)$. Conditioned on this grounded prompt, the LLM generates an output token sequence $\boldsymbol{\nu} = (\nu_1, \dots, \nu_{N_{\text{tok}}})$ according to the autoregressive conditional distribution $p(\boldsymbol{\nu} \mid \mathbf{p}(\xi))$ given by [38]

$$p(\boldsymbol{\nu} \mid \mathbf{p}(\xi)) = \prod_{\varpi=1}^{N_{\text{tok}}} p(\nu_{\varpi} \mid \nu_{<\varpi}, \mathbf{p}(\xi)), \quad (34)$$

where ϖ denotes the token-position index, $\nu_{<\varpi} = (\nu_1, \dots, \nu_{\varpi-1})$ denotes the previously generated token prefix, and N_{tok} is the generated sequence length.

In the proposed framework, the main roles of RAG are to minimize hallucinations [41] and to provide reliable domain grounding for formulation. Specifically, RAG helps the LLM identify the key system components in the current CMfg scenario, such as UAVs, manufacturing stations, ISDs, computational tasks, and the central depot with cloud support, together with their interactions. It also helps the LLM determine relevant execution modes, routing-offloading couplings, resource and energy constraints, and objective components. Consequently, RAG constrains the search space of the LLM, enhances alignment with domain knowledge, and mitigates unsupported or fabricated outputs. The next chain-of-thought module subsequently arranges the acquired knowledge into a cohesive mathematical model.

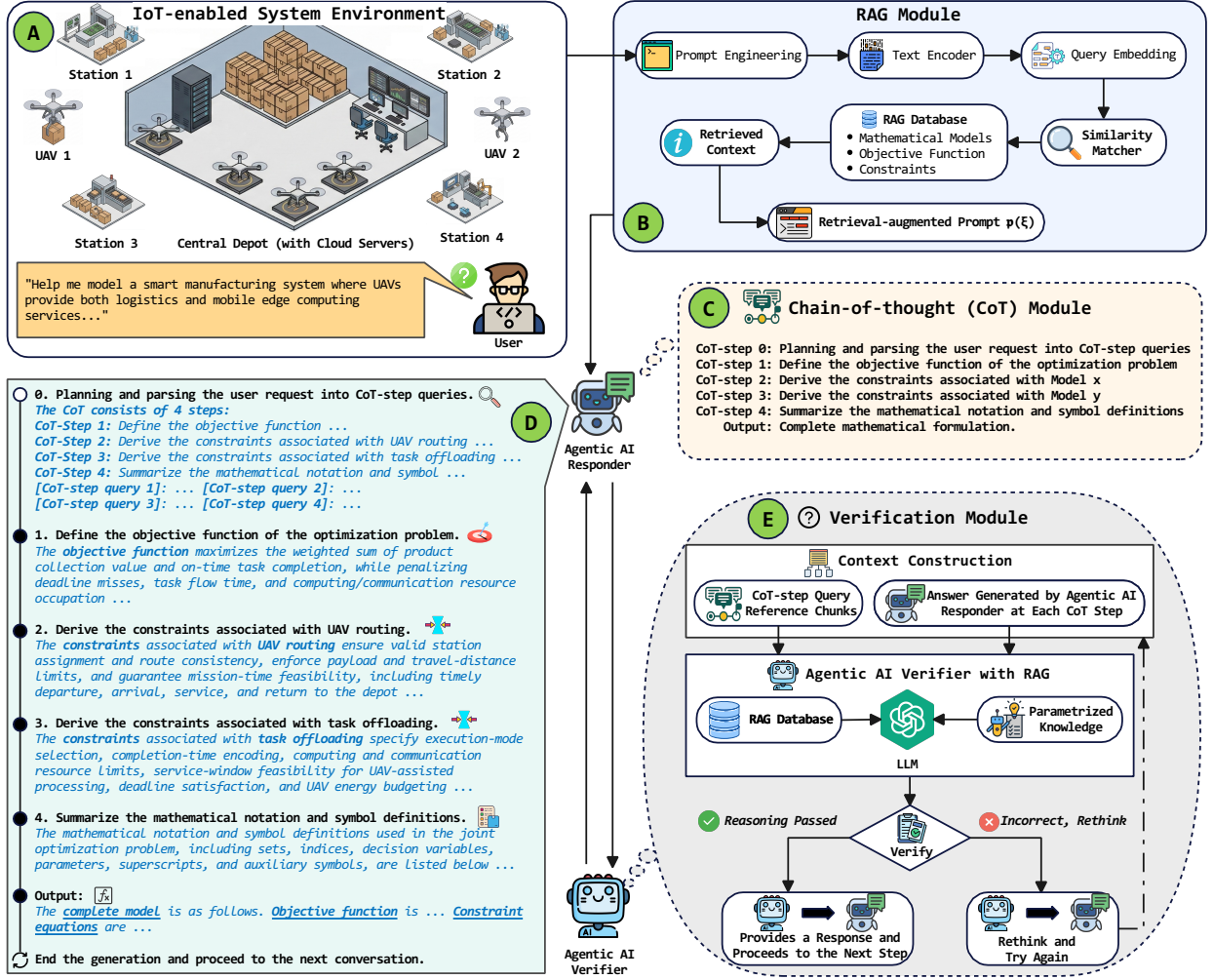


Fig. 2. An overview of the proposed agentic AI framework. *Part A* presents the IoT-enabled system environment and the user’s natural-language modeling request. *Part B* illustrates the RAG module for retrieving modeling knowledge and building a retrieval-augmented prompt. *Part C* outlines the CoT module for step-by-step formulation reasoning. *Part D* corresponds to the *Agentic AI Responder* that generates the formulation outputs. *Part E* depicts the verification module, where the *Agentic AI Verifier* checks each CoT-step output and either permits continuation or triggers rethinking.

B. Chain-of-Thought and Verification

After retrieval grounds the LLM with domain-relevant evidence, the CoT module reasons over the retrieved information and organizes it for downstream mathematical formulation. Recent Long-CoT studies characterize this reasoning paradigm by deep reasoning, extensive exploration, and feasible reflection, which support more intricate and coherent reasoning processes [42]. In Fig. 2, this module externalizes the intermediate reasoning that links the problem description to the objective terms and constraint families, thereby making the formulation process more transparent, in line with recent work on interactive reasoning and explicit reasoning-chain manipulation [43]. Recent studies further show that verifier-guided CoT can assess or guide reasoning-step correctness, while verifiability-oriented evaluation can assess reasoning quality beyond final-answer accuracy [44], [45]. The CoT-based formulation step is particularly important in the CMfg scenario considered here, where UAV routing and computational task offloading are tightly coupled through time-varying service availability,

communication and computing limits, energy budgets, and task deadlines.

Given a user request ξ and the retrieval-augmented prompt $p(\xi)$, the LLM performs a structured reasoning process over the retrieved evidence. The framework in Fig. 2 includes two agents: the *Agentic AI Responder* and the *Agentic AI Verifier*. The *Agentic AI Responder* is responsible for reasoning over the user request and providing the final answer. In contrast, the *Agentic AI Verifier* evaluates another agent’s responses, deciding whether to approve or reject it. The CoT module of the *Agentic AI Responder* is shown in *Part C* of Fig. 2. The CoT module first analyzes the user query and decomposes it into a sequence of CoT-step queries, so that the original request can be solved in a structured, step-by-step manner. Specifically, during the initialization phase, an LLM parses the user query and generates both a CoT-step plan and the corresponding CoT-step queries for the *Agentic AI Responder*. These CoT-step queries define the reasoning order and specify what the responder should address at each stage [46].

As shown in *Part D* of Fig. 2, the user request is decom-

posed into four steps, namely objective function formulation, UAV routing constraints derivation, task offloading constraints derivation, and mathematical notation summarization. During execution, the *Agentic AI Responder* answers the CoT-step queries sequentially [47]. For example, in CoT-step 1, the query on defining the objective function is first sent to the responder, which then produces an initial response. This response is subsequently forwarded to the *Agentic AI Verifier* for assessment through the verification module. If the response does not pass verification, the verifier returns feedback requiring the responder to rethink and revise the current answer. In contrast, if the response is verified as correct, the workflow proceeds to the next CoT-step query [48]. The workflow then proceeds in the same manner for the remaining CoT steps. CoT-step 2 derives the constraints associated with UAV routing. CoT-step 3 focuses on the constraints for task offloading. CoT-step 4 summarizes the mathematical notation and symbol definitions used in the formulation.

For each CoT step, the *Agentic AI Responder* initially formulates a response to the specific CoT-step query; subsequently, the *Agentic AI Verifier* assesses the generated response prior to the continuation of the reasoning process [48]. This sequential reasoning-and-verification methodology ensures the system's stepwise completion of the entire CoT workflow, with the final output being generated only subsequent to the successful verification of all CoT-step responses. Upon the successful verification of all CoT-step responses, the system consolidates the verified intermediate outputs into a comprehensive final response for the user. Consequently, from the user's perspective, the overall workflow can be viewed as a direct mapping from the input query to a complete mathematical model. The intermediate reasoning, decomposition, and verification procedures are handled internally by the CoT module in *Part C* and the verification module in *Part E*. Together, these two modules realize an automated multi-agent reasoning-and-verification process between the *Agentic AI Responder* and the *Agentic AI Verifier*.

Part E of Fig. 2 further presents the verification module of the *Agentic AI Verifier*. For each CoT-step query, the relevant reference chunks are obtained from the RAG database and combined with the corresponding response generated by the *Agentic AI Responder* to construct the verification context [49]. Based on this context, together with parameterized knowledge, the verifier determines whether the responder's answer at the current CoT step is semantically correct and logically consistent. If the answer passes verification, the workflow continues to the next CoT step; otherwise, the verifier instructs the responder to rethink and try again until either a satisfactory response is obtained or the maximum number of allowed retries is reached, at which point the verification loop is terminated to avoid an unbounded iteration process.

VI. PROPOSED HIERARCHICAL DRL APPROACH

Inspired by [8], we decompose the joint UAV routing and MEC problem into a hierarchical two-layer framework, which reduces the state and action spaces while preserving the key coupling between logistics and computation. Both layers are

solved using PPO-based DRL. Throughout this paper, terrestrial MEC servers are referred to as the “cloud”.

A. Hierarchical Framework Overview

The proposed hierarchical framework consists of two sequential PPO-based DRL layers. The upper layer solves the multi-UAV routing problem as a centralized MDP to determine visiting sequences and station assignments under energy, capacity, and flight distance constraints. Conditioned on the upper-layer routes, the lower layer optimizes per-slot task execution by jointly deciding offloading destinations and communication/computing resource allocation under service-window, capacity, and deadline constraints. To couple the two layers, the upper layer provides the lower layer with each UAV's service window and remaining energy headroom. Each service window gives the arrival and departure times of a UAV at a station. In the co-training procedure, the upper layer is first trained, after which its best policy is used to initialize the lower-layer training.

B. Upper-layer DRL: MDP Design

The upper layer governs the coarse-timescale routing decisions of U UAVs tasked with visiting M stations within a mission horizon of duration T_{mission} . Each upper-layer training cycle consists of multiple complete routing episodes, where each episode comprises a sequence of discrete *upper steps*. During an upper step, routing decisions are taken, UAV states evolve according to physical constraints, and rewards are accumulated.

1) *Upper-layer MDP*: The routing problem at the upper layer is modeled as a finite-horizon Markov decision process (MDP) with the following components

$$\mathcal{M}^{\text{up}} = (\mathcal{S}^{\text{up}}, \mathcal{A}^{\text{up}}, \mathcal{R}^{\text{up}}, \gamma^{\text{up}}),$$

where $\mathcal{S}^{\text{up}}$, $\mathcal{A}^{\text{up}}$, $\mathcal{R}^{\text{up}}$, and γ^{up} are the state space, action space, reward function, and discount factor of upper-layer DRL, respectively.

2) *State space*: At each upper step t_a , the state $s_{t_a}^{\text{up}} \in \mathcal{S}^{\text{up}}$ is a continuous vector that summarizes four types of information: (i) per-UAV status, including location, remaining energy, payload, and flight-distance budget; (ii) route-progress information, including normalized route length, completion status, and the set of stations already served by each UAV; (iii) per-station attributes, including service status, normalized value and payload weights, and local service characteristics; and (iv) mission-level context, including normalized mission time, the fraction of completed collections, and UAV-station distance information for feasibility and planning.

3) *Action space*: At each upper step, the learned upper-layer policy selects a joint action $a_{t_a}^{\text{up}}$ defined as:

$$a_{t_a}^{\text{up}} = (a_{t_a}^{\text{prio}}, a_{t_a}^{\text{route}}) \in \mathcal{A}^{\text{up}}, \quad (35)$$

where $a_{t_a}^{\text{prio}}$ refers to the priority score [50], [51], and is a real-valued vector whose dimension equals the number of UAVs and whose ranking determines the order in which the UAVs

apply their routing decisions during the current upper step. $a_{t_a}^{\text{route}}$ is the routing actions [52]. For each UAV u , the learned upper-layer policy selects one discrete routing symbol from

$$a_{t_a}^{\text{route}}(u) \in \{\text{stay}, \text{add}(1), \dots, \text{add}(M), \text{complete}\}, \quad (36)$$

where *stay* keeps the UAV at its current location. *add*(m) instructs the UAV to visit station m until M stations, while *complete* returns the UAV to the depot and closes its current route. The routing decisions and priority scores constitute the actionable degrees of freedom. Feasibility masks restrict the learned upper-layer policy to physically admissible routing symbols. An action such as *add*(m) becomes infeasible if its flight distance, return distance, payload, minimum service time, or remaining energy would violate safety margins.

4) *Reward function*: At upper step t_a , conditioned on the observed state $s_{t_a}^{\text{up}}$, the learned upper-layer policy outputs action $a_{t_a}^{\text{up}}$, and the environment transitions to $s_{t_a+1}^{\text{up}}$, where $t_a \in \{0, 1, \dots, T^{\text{up}} - 1\}$, and T^{up} is the number of executed upper steps in the episode. The per-step reward, $r_{t_a}^{\text{up}}$, is defined as:

$$r_{t_a}^{\text{up}} = \sum_{m \in \mathcal{C}_{t_a}} v_m + b_{\text{cov}} + b_{\text{bal}} + b_{\text{del}} - \kappa_{\text{con}} + \psi_{\text{end}}^{\text{up}}. \quad (37)$$

The term $\sum_{m \in \mathcal{C}_{t_a}} v_m$ is the collection value obtained during the transition, where $\mathcal{C}_{t_a}$ denotes the set of stations newly collected at upper step t_a , and v_m is the product value of station m . The term b_{cov} is a shaping reward that encourages increasing the coverage ratio of collected stations, b_{bal} encourages balanced route workloads across UAVs, and b_{del} rewards route termination according to two factors: the collected route value and the ratio between the collected route value and the closed-tour distance. The term κ_{con} is an aggregated penalty that discourages infeasible decisions, including violations of energy, payload, remaining-distance, duplicate-visit, movement-time, service-time, and mission-completion constraints. The last term $\psi_{\text{end}}^{\text{up}}$ is the terminal-condition term, activated only at the final upper step. It adjusts the final reward according to the terminal outcome by granting a success bonus or imposing a penalty based on the number of unserved stations.

C. Information Flow from the Upper Layer to the Lower Layer

Each completed upper-layer routing episode provides two types of information for subsequent lower-layer training cycles: the feasible service windows extracted for each UAV-station pair from the corresponding mission-time intervals, and the residual energy headroom of all UAVs at the end of the episode, which is used to scale certain lower-layer resource and capacity conditions.

D. Lower-layer DRL: MDP Design

In each time slot, the lower-layer DRL scheduler performs task scheduling and resource allocation for computation tasks generated by multiple ISDs. Each task can be executed locally, offloaded to a UAV (edge computing), or offloaded to the cloud via a selected UAV. Tasks may span multiple slots. The

lower-layer DRL scheduler seeks to accelerate task progress and completion, reduce backlog, and avoid deadline misses, subject to per-slot compute/communication capacity constraints and time-varying UAV service-window constraints determined by the upper layer.

At the beginning of the time slot, the environment collects all unfinished tasks, including (i) waiting tasks and (ii) active tasks that have started but are not yet completed. These tasks are consolidated into a global queue and prioritized according to urgency using the earliest-deadline-first (EDF) policy. The lower-layer DRL scheduler then scans the global queue from most urgent to least urgent and selects up to a preset maximum number of tasks to process in this slot. For each selected task, it first chooses exactly one execution location (local / a specific UAV / cloud via a specific UAV) and then assigns discrete compute and communication resource levels to advance task processing. The aggregated allocations across all selected tasks must remain feasible under the capacity and service-window constraints.

1) *Lower-layer MDP*: The lower-layer scheduling problem is modeled as an MDP defined by the tuple:

$$\mathcal{M}^{\text{lo}} = (\mathcal{S}^{\text{lo}}, \mathcal{A}^{\text{lo}}, \mathcal{R}^{\text{lo}}, \gamma^{\text{lo}}),$$

where $\mathcal{S}^{\text{lo}}$, $\mathcal{A}^{\text{lo}}$, $\mathcal{R}^{\text{lo}}$, and γ^{lo} are the state space, action space, reward function, and discount factor of lower-layer DRL, respectively.

2) *State space*: The lower-layer state $s_{t_b}^{\text{lo}} \in \mathcal{S}^{\text{lo}}$ is observed at the beginning of slot t_b and summarizes local queues, UAV-side resources, and current service feasibility through six groups of normalized features: (i) per-ISD local statistics, including local compute capability and headroom, queue and active-local workload summaries, and the minimum remaining time-to-deadline; (ii) per-UAV statistics, including energy headroom, compute and communication capacities, and workload summaries of tasks currently executed on the UAV or relayed to the cloud via that UAV; (iii) global system scalars, such as the current and remaining time, aggregated counts of waiting and active tasks across processing modes, and a summary statistic of UAV energy; (iv) per UAV-ISD active offload status, represented by normalized aggregates of remaining workload and communication bits for active tasks from ISD k associated with UAV u ; (v) a UAV-ISD service-window availability mask indicating whether UAV u can serve ISD k at time t_b ; and (vi) a fixed-length, zero-padded snapshot of the top- N_q unfinished tasks ordered by serviceability and urgency, with per-task attributes including origin, status, execution mode and UAV index when applicable, remaining time-to-deadline, and remaining compute and communication requirements.

3) *Action space*: At the beginning of each slot t_b , the learned lower-layer policy outputs a joint action defined over a cached global-queue snapshot Q_{t_b} . This joint action specifies the service decisions for the first K_g queue positions. K_g is a fixed truncation size and only the first $\min\{K_g, |Q_{t_b}|\}$ entries take effect. The top- K_g is different from the top- N_q queue snapshot used in the state. N_q denotes the number of highest-priority unfinished tasks encoded in the state. K_g denotes the number of highest-priority queue positions on which the

learned lower-layer policy outputs per-slot decisions. The action is given by

$$A_{t_b}^{lo} = \left\{ (\ell_{t_b}^{(j)}, \tilde{\alpha}_{t_b}^{(j)}, \tilde{\beta}_{t_b}^{(j)}) \right\}_{j=1}^{K_g}, \quad (38)$$

where the j -th tuple gives the processing decision and resource-allocation scalars for the task at position j in Q_{t_b} . The discrete variable $\ell_{t_b}^{(j)}$ specifies a processing choice for the queue item at position j . It is selected from the semantic action set:

$$\ell_{t_b}^{(j)} \in \{\text{SKIP}, \text{local}\} \cup \{\text{UAV}(u), \text{cloud-via-UAV}(u)\}_{u=1}^U, \quad (39)$$

where SKIP denotes no new dispatch, while the other options correspond to local execution, UAV execution, or cloud execution via UAV u . The SKIP option allows the learned lower-layer policy to serve fewer tasks in a slot when needed to respect resource limits and prioritize more urgent or more serviceable tasks.

The scalars $\tilde{\alpha}_{t_b}^{(j)}, \tilde{\beta}_{t_b}^{(j)} \in [0, 1]$ denote normalized compute and communication allocations, which are quantized by the environment into discrete levels. The continuous sampling of $\tilde{\alpha}_{t_b}^{(j)}$ and $\tilde{\beta}_{t_b}^{(j)}$ is adopted to avoid a high-dimensional multi-discrete action space, while quantization ensures feasibility under level-based resource constraints. For local execution, communication allocation is ignored. For active tasks already bound to a processing mode and UAV index, $\ell_{t_b}^{(j)}$ is restricted to either SKIP or the previously assigned processing option. If selected, the allocation is updated for the current slot; otherwise, previous allocations persist.

4) *Reward function*: The step reward $r_{t_b}^{lo}$ is designed to align with the objectives: (i) finish tasks as early as possible, (ii) strictly avoid deadline failures, and (iii) discourage avoidable idling and excessive (always-max) resource occupation. A dense-and-aligned reward is written as:

$$\begin{aligned} r_{t_b}^{lo} = & b_{\text{complete}} + b_{\text{dispatch}} + b_{\text{progress}} \\ & + \Phi_B + \Phi_U - \kappa_{\text{deadline}} - \kappa_{\text{invalid}} \\ & - \kappa_{\text{idle}} - \kappa_{\text{alloc}} - \kappa_{\text{living}} + \psi_{\text{end}}^{lo}. \end{aligned} \quad (40)$$

At slot t_b , the lower-layer reward $r_{t_b}^{lo}$ combines positive incentives, penalties, and an end-of-episode settlement. Specifically, b_{complete} rewards task completions in slot t_b , b_{dispatch} assigns a bonus or cost to each waiting task dispatched in that slot according to its selected execution destination, e.g., local, UAV, or cloud via a UAV. b_{progress} provides a dense reward for effective task advancement during the slot. The shaping terms Φ_B and Φ_U further encourage backlog reduction and urgency reduction, respectively, by rewarding transitions that reduce the waiting-task backlog and the urgency of unfinished tasks. In contrast, κ_{deadline} penalizes deadline violations, κ_{invalid} penalizes infeasible decisions that violate service-window or resource-capacity constraints, and κ_{idle} penalizes avoidable “no-dispatch” behavior when feasible waiting tasks exist. In addition, κ_{alloc} discourages excessive resource usage by penalizing the aggregate normalized allocation levels of computation and communication resources, and κ_{living} imposes a persistent penalty on the remaining unfinished load after the current transition so as to maintain continuous pressure

for prompt task clearance. Finally, ψ_{end}^{lo} denotes the terminal contribution applied at the end of the episode, which penalizes unfinished tasks remaining at termination, imposes a stronger penalty on overdue tasks, and may also provide a sparse bonus when the system is cleared early after the final task arrival.

E. Complexity Analysis of Upper- and Lower-Layer DRL

Let N_{up} and N_{lo} denote the numbers of training episodes for the upper and lower layers, respectively, and let $\bar{T}^{\text{up}}$ and $\bar{T}^{\text{lo}}$ denote the average episode lengths. Let $P_{\theta, \phi}^{\text{up}}$ and $P_{\theta, \phi}^{\text{lo}}$ denote the total numbers of trainable parameters in the upper-layer and lower-layer actor-critic networks, respectively. In the upper layer, routing-action evaluation and feasibility-mask construction scale with the number of UAV-station pairs. Thus, the upper-layer training complexity is $\mathcal{O}(N_{\text{up}} \bar{T}^{\text{up}} (UM + P_{\theta, \phi}^{\text{up}}))$, under fixed PPO update settings. In the lower layer, the state contains per-ISD features, per-UAV features, UAV-ISD service-window masks, and a top- N_q queue snapshot, while the policy only acts on the first K_g queue positions. Hence, the lower-layer training complexity is $\mathcal{O}(N_{\text{lo}} \bar{T}^{\text{lo}} (UK + N_q + K_g U + P_{\theta, \phi}^{\text{lo}}))$. A monolithic MDP would need to enumerate joint routing and task-scheduling choices, whose size scales with $(M + 2)^U (2U + 2)^{K_g}$ before considering resource-allocation levels. Therefore, the proposed hierarchical decomposition keeps the rollout and update costs polynomial in the main system sizes while preserving the service-window coupling between UAV routing and MEC task scheduling.

F. Training Procedure

The two-layer DRL framework is trained sequentially. First, the upper layer is trained by PPO for UAV routing, where routing rollouts are collected under feasibility masks and the actor-critic networks are updated whenever the rollout buffer is full. The best-performing upper-layer policy is then fixed to extract the UAV service-window and residual-energy information for the lower layer. Next, the lower layer is trained by PPO for task scheduling and resource allocation under this fixed routing information. In this stage, scheduling rollouts are collected over EDF-based queue snapshots with feasibility masks and quantized allocation decisions, and the actor-critic networks are updated whenever the rollout buffer is full. This sequential design preserves the cross-layer coupling between routing and MEC scheduling while avoiding simultaneous optimization, thereby improving training stability.

VII. PERFORMANCE EVALUATION

A. Simulation Parameters and Setup

In this subsection, we describe the scenario settings, agentic AI settings, and DRL settings as follows.

1) *Scenario Settings*: We consider a cloud-manufacturing scenario with two homogeneous UAVs serving six manufacturing stations, where each station is equipped with one ISD. The mission horizon is set to 313 s. The minimum required service durations are set to 100 s at the first station, 80 s at the second, third, and fourth stations, and 70 s at the fifth and

TABLE III
SCENARIO SETTINGS

System Parameter	Value
UAV flight speed	10 m/s
Maximum travel distance	4800 m
UAV payload capacity	30 payload-units
UAV battery capacity	150 Wh
Local computing capacity	300 work-units/s
UAV computing capacity	400 work-units/s
Cloud computing capacity	400 work-units/s
ISD-to-UAV / UAV-to-cloud rate	500/120 Mb/s

sixth stations. Here, payload-units denote an abstract unit used to measure product weight and UAV payload budget. For MEC scheduling, the lower layer is discretized into 1-s time slots, and work-units are used as an abstract unit to measure computation workload and processing capacity. In each episode, 1920 tasks are randomly generated, with deadlines uniformly distributed from 20 s to 80 s. The task workload is randomly generated around 10 work-units per task, with a typical deviation of about 2 work-units, while the task input size is randomly generated around 10^5 bits per task, with a typical deviation of about 2×10^4 bits. The other scenario settings are listed in Table III.

2) *Agentic AI Settings*: The agentic AIs are implemented as follows. First, the user description and expertise knowledge are encoded using OpenAIEmbeddings with *text-embedding-ada-002* [53]. Next, a local RAG pipeline is constructed by indexing the text corpus into a Chroma vector store [54] and exposing a similarity-based retriever as a retrieval tool. The planning, response generation, and verification components all employ *gpt-5.4-2026-03-05* [55]. Finally, the overall reasoning process is organized as a two-agent CoT workflow, consisting of a Responder Agent and a Verifier Agent, while short-term conversational memory across successive turns is maintained using LangGraph checkpointing [56] within the LangChain-based agent framework [57].

3) *DRL Settings*: We adopt a hierarchical DRL framework for the considered hybrid logistics–MEC scheduling problem. In both compared schemes, the upper layer uses PPO to optimize UAV routing, while the lower layer is used to optimize task execution and resource allocation. To evaluate the effectiveness of our proposed approach, we compare it with a baseline method using advantage actor-critic (A2C). The baseline scheme employs PPO in the upper layer and A2C in the lower layer, whereas the proposed scheme employs PPO in both the upper and lower layers. For both layers, the actor and critic networks use two hidden layers with 256 neurons per layer. ReLU is adopted as the activation function, and Adam is used for network optimization. The detailed hyperparameter settings of the upper-layer PPO (Up-PPO), lower-layer PPO (Low-PPO), and lower-layer A2C (Low-A2C) are summarized in Table IV.

B. Effectiveness of the Agentic AI

Fig. 3 presents the results of the proposed agentic AI framework, comprising the *Agentic AI Responder* and *Agentic*

TABLE IV
DRL HYPERPARAMETER SETTINGS

System Parameter	Up-PPO	Low-PPO	Low-A2C
Learning rate (actor)	2×10^{-4}	7×10^{-5}	1×10^{-5}
Learning rate (critic)	3×10^{-4}	1×10^{-4}	5×10^{-5}
Discount factor	0.99	0.996	0.996
GAE / trace parameter	0.92	0.95	0.95
Clipping parameter	0.15	0.05	–
Entropy coefficient	0.004	0.001	0.001
Rollout length	96	512	64
Batch size	32	512	–
Training episodes	2000	2000	2000
Number of hidden layers	2	2	2
Hidden layer size	256	256	256
Activation function	ReLU	ReLU	ReLU
DNN optimizer	Adam	Adam	Adam

Abbreviations: Generalized advantage estimation (GAE); deep neural network (DNN).

AI Verifier, along with a single-agent baseline that includes only the *Agentic AI Responder* without CoT and verification modules. In *Part A* of Fig. 3, the user first provides the prompts, including the system description and the required mathematical formulation. A CoT planning prompt, shown in *Part B*, is then sent to the proposed framework in *Part C*. Finally, the *Agentic AI Responder*, equipped with both CoT and verification modules, generates the complete response, of which only a partial example is presented in *Part D*. In addition, a single-agent baseline is shown in *Part E*. It consists only of the *Agentic AI Responder* equipped with RAG and does not include either the CoT module or the verification module. *Part F* presents several examples extracted from its complete response.

In Fig. 3, *Part D* and *Part F* present examples generated by the proposed agentic AI framework and the baseline, respectively. The main difference between them lies in their semantic fidelity to the RAG database. The proposed framework remains closer to the original modeling logic by preserving the intended meanings of the modeled terms, whereas the baseline tends to reformulate them into a different optimization-oriented representation. Therefore, the distinction is not merely syntactic, but conceptual. For example, the semantic difference is reflected in what the resource-related penalty is intended to measure. In the RAG database, the penalty is defined through the abstract occupation variables R_t^{cmp} and R_t^{com} , which represent normalized computing and communication occupation. The proposed framework preserves this same interpretation, so the penalty still refers to an aggregated and normalized notion of overall resource occupation. By contrast, the baseline rewrites this penalty as a direct sum of computing and communication allocation variables. As a result, it changes the penalty from normalized resource occupation to raw resource usage. This is a semantic change, not merely a notational difference.

C. Effectiveness of the Hierarchical DRL Approach

The effectiveness of the upper-layer and lower-layer DRL approach are discussed as follows.

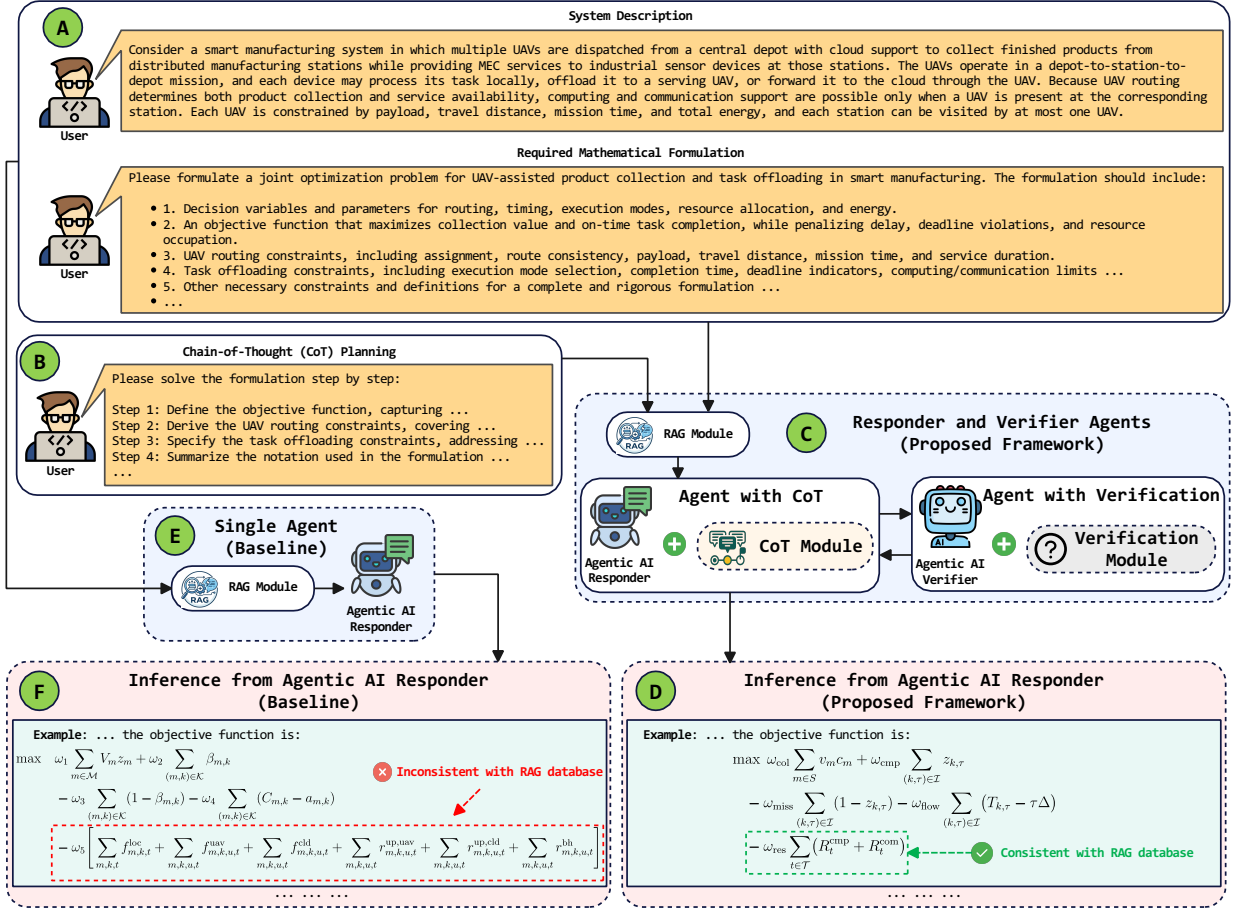


Fig. 3. Results from the proposed agentic AI framework. *Part A* presents an example of a user's system description and mathematical formulation request. *Part B* illustrates the CoT planning from the user. *Part C* outlines the conversation between the responder and verifier agents. *Part D* lists the examples from the final answer of *Agentic AI Responder* with CoT module and verification module. *Part E* depicts a single agent framework. *Part F* lists the examples from the final answer of *Agentic AI Responder* without CoT module and verification module.

1) Upper-layer DRL Training Results: In Fig. 4 (a), the upper-layer PPO shows good learning ability and convergence. *Raw* denotes the reward of each episode and *MA(50)* is the moving average of the reward of the last 50 episodes, which can better reflect the overall training trend. In the early stage, the *Raw* reward fluctuates sharply, indicating active exploration. With training, *MA(50)* increases rapidly and then stabilizes around 400, indicating that the policy is improved rapidly and converges to a stable high reward solution. Although occasional drops still appear in the *Raw* reward, the stable *MA(50)* suggests that the overall training performance is robust. Fig. 4 (b) further shows that the learned upper-layer policy achieves a high collection rate. After several early exploratory fluctuations, the collection rate quickly approaches 100% and remains at or near 100% in most episodes, indicating that the PPO policy can reliably plan UAV routing and complete almost all collection tasks. Although a few occasional drops still appear, the overall result remains highly stable.

2) Lower-layer DRL Training Results: Fig. 5 compares the lower-layer DRL training performance of PPO and A2C in terms of total reward and deadline satisfaction rate, where the deadline satisfaction rate denotes the fraction of tasks completed within their deadlines in each episode. Fig. 5 (a) shows that

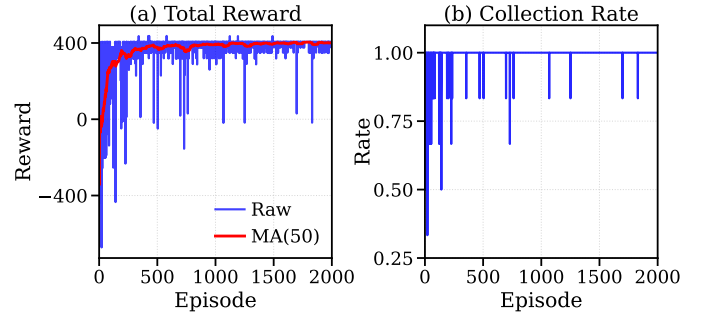


Fig. 4. Upper-layer DRL training results with PPO: Total rewards and collection rate.

both PPO and A2C improve the lower-layer policy from highly negative rewards to a converged positive-reward region. PPO exhibits larger fluctuations in the intermediate stage, but its reward quickly recovers and remains relatively stable afterward. In contrast, the total reward under A2C increases more smoothly in the early stage, but several sharp reward drops still appear in the later stage, indicating weaker stability after convergence. Fig. 5 (b) shows that both methods achieve a high deadline satisfaction rate after training. However, PPO maintains a

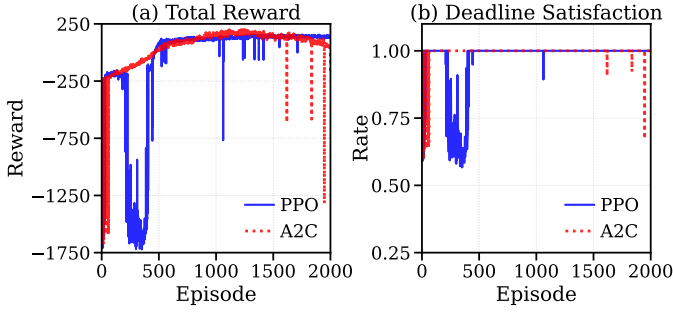


Fig. 5. Lower-layer DRL training results with PPO and A2C: Total rewards and deadline satisfaction rate.

rate very close to 100% more consistently in the later stage, whereas A2C still experiences several late-stage drops and cannot always sustain the ideal 100% deadline satisfaction rate. Based on these results, PPO is preferred for the lower-layer DRL training. The main reason is that PPO provides more robust converged performance, with more stable rewards and more reliable deadline satisfaction in the later training stage. In particular, unlike A2C, PPO is able to maintain nearly perfect deadline satisfaction in most converged episodes.

VIII. CONCLUSION

In this paper, we have studied a hybrid coordination problem in CMfg, where UAV-assisted product collection is tightly coupled with MEC task processing for industrial sensor devices. To address the difficulty of formulating such a problem, we have proposed an interactive agentic AI framework that integrates LLMs, RAG, and CoT reasoning to support interpretable mathematical modeling. To solve the resulting optimization problem, we have further developed a hierarchical DRL approach based on PPO, in which the upper layer handles UAV routing and the lower layer performs computational task scheduling and resource allocation under coupled operational constraints. Simulation results have demonstrated the effectiveness of the proposed framework. The learned policies can achieve strong routing and scheduling performance, while the overall design preserved the essential coupling between logistics and computation through compact cross-layer information exchange. Overall, this work provides a unified framework for both formulating and solving hybrid logistics-computation scheduling problems in CMfg. Future work can extend the framework to larger-scale systems, heterogeneous UAV fleets, and more dynamic manufacturing environments.

IX. APPENDIX

A. Dataset Construction

To validate our proposed framework, the RAG dataset is constructed using Eq. (28) as the optimization objective, where the UAV routing model is subject to the constraints from Eq. (1) to Eq. (9), and the computational task offloading model follows the constraints from Eq. (19) to Eq. (27). Our code and dataset for the proposed agentic AI framework are uploaded to GitHub ².

²<https://github.com/Puppet88/Agentic-AI-UAV>

REFERENCES

- [1] Z. Wang, C. Wan, J. Liu *et al.*, “Masc: Large language model-based multi-agent scheduling chain for flexible job shop scheduling problem,” *Advanced Engineering Informatics*, vol. 67, p. 103527, 2025.
- [2] W. Gu, Y. Cao, Y. Li *et al.*, “Large language model-empowered dynamic scheduling for intelligent hybrid flow shop using multi-agent deep reinforcement learning,” *Advanced Engineering Informatics*, vol. 71, p. 104294, 2026.
- [3] Z. Huang, L. Guo, J. Sheng *et al.*, “Graphthought: Graph combinatorial optimization with thought generation,” *arXiv:2502.11607*, 2025.
- [4] H. Abgaryan, A. Harutyunyan, and T. Cazenave, “Llms can schedule,” *arXiv:2408.06993*, 2024.
- [5] Z. Zhao, D. Tang, H. Zhu *et al.*, “A large language model-based multi-agent manufacturing system for intelligent shopfloor,” *arXiv:2405.16887*, 2024.
- [6] L. Bo-hu, Z. Lin, W. Shi-long *et al.*, “Cloud manufacturing: a new service-oriented networked manufacturing model,” *Computer Integrated Manufacturing System*, vol. 16, no. 01, pp. 0–0, 2010.
- [7] X. Wang, Y. Laili, L. Zhang *et al.*, “Hybrid task scheduling in cloud manufacturing with sparse-reward deep reinforcement learning,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1878–1892, 2025.
- [8] X. Mao, G. Wu, M. Fan *et al.*, “DI-drl: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-uav,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1028–1044, 2025.
- [9] Y. Li, Q. Wang, X. Li *et al.*, “Real-time scheduling for flexible job shop with agvs using multiagent reinforcement learning and efficient action decoding,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 55, no. 3, pp. 2120–2132, 2025.
- [10] G. Sun, J. Wu, Z. Sun *et al.*, “JC⁵a: Service delay minimization for aerial mec-assisted industrial cyber-physical systems,” *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 2976–2993, 2025.
- [11] L. Jiao, L. Gao, J. Zheng *et al.*, “Optimizing 3d trajectory and task offloading in collaborative uav-enabled mobile edge computing networks,” *Computer Networks*, vol. 282, p. 112283, 2026.
- [12] W. P. Coutinho, J. Fliege, M. Battarra *et al.*, “Routing a fleet of unmanned aerial vehicles: A trajectory optimisation-based framework,” *Transportation Research Part B: Methodological*, vol. 200, p. 103312, 2025.
- [13] G. O. Chagas, L. C. Coelho, D. Laganà *et al.*, “A dynamic drone routing problem with uncertain demand and energy consumption,” *Transportation Research Part B: Methodological*, vol. 202, p. 103335, 2025.
- [14] F. WANG, H. ZHANG, S. DU *et al.*, “C-sppo: A deep reinforcement learning framework for large-scale dynamic logistics uav routing problem,” *Chinese Journal of Aeronautics*, vol. 38, no. 5, p. 103229, 2025.
- [15] Y. Li, S. Wang, H. Sun *et al.*, “Collaborative vessel-unmanned aerial vehicle routing for time-window-constrained offshore parcel delivery,” *Transportation Research Part C: Emerging Technologies*, vol. 178, p. 105189, 2025.
- [16] M. Wang, S. Chen, and Q. Meng, “Drone routing problem for shore-to-ship delivery services considering non-linear energy consumption,” *Transportation Research Part B: Methodological*, vol. 206, p. 103410, 2026.
- [17] X. Chen, J. Cao, Y. Sahni *et al.*, “Mobility-aware dependent task offloading in edge computing: A digital twin-assisted reinforcement learning approach,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 2979–2994, 2025.
- [18] C. Dong, W. Li, Z. Zhou *et al.*, “Delay-sensitive task offloading with edge caching through martingale-based deep reinforcement learning,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 7, pp. 6225–6242, 2025.

- [19] X. Chen, J. Cao, R. Cao *et al.*, “Decentralized task offloading in collaborative edge computing: A digital twin assisted multi-agent reinforcement learning approach,” *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 4776–4790, 2026.
- [20] A. Nabi and S. Moh, “Joint offloading decision, user association, and resource allocation in hierarchical aerial computing: Collaboration of uavs and hap,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 8, pp. 7267–7282, 2025.
- [21] H. Gu, L. Zhao, Z. Han *et al.*, “Joint task offloading and resource allocation in ultra-dense multi-access edge computing: A mean field learning approach,” *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3598–3615, 2026.
- [22] H. Zhang, R. Zhang, W. Zhang *et al.*, “Advancing generative artificial intelligence and large language models for demand side management with internet of electric vehicles,” *IEEE Internet of Things Journal*, pp. 1–1, 2026.
- [23] Y. Ren, Y. Liu, T. Ji *et al.*, “Ai agents and agentic ai—navigating a plethora of concepts for future manufacturing,” *Journal of Manufacturing Systems*, vol. 83, pp. 126–133, 2025.
- [24] J. Lee and H. Su, “Agentic ai for smart manufacturing,” *Manufacturing Letters*, vol. 46, pp. 92–96, 2025.
- [25] L.-C. Chen, M. S. Pardeshi, Y.-X. Liao *et al.*, “Application of retrieval-augmented generation for interactive industrial knowledge management via a large language model,” *Computer Standards & Interfaces*, vol. 94, p. 103995, 2025.
- [26] M. Li, Q. Zhou, W. Li *et al.*, “A4ps: Agentic ai-assisted advanced planning and scheduling with large language models for smart manufacturing,” *Journal of Manufacturing Systems*, vol. 85, pp. 207–226, 2026.
- [27] N. M. Farid, A. Taghizadeh, and S. Shafiee, “Agentic data analysis for intelligent manufacturing: Benchmark-driven evaluation of agentic vs. direct llm approaches,” *Procedia CIRP*, vol. 139, pp. 280–285, 2026, 13th CIRP Global Web Conference.
- [28] S. Li, T. Liao, G. Wu *et al.*, “Drone on-demand delivery routing problem considering order splitting and battery swapping,” *Computers & Industrial Engineering*, vol. 208, p. 111388, 2025.
- [29] S. I. Satoglu and I. E. Sahin, “Design of a just-in-time periodic material supply system for the assembly lines and an application in electronics industry,” *The International Journal of Advanced Manufacturing Technology*, vol. 65, no. 1, pp. 319–332, 2013.
- [30] D. Askerbekov, J. A. Garza-Reyes, R. Roy Ghatak *et al.*, “Embracing drones and the internet of drones systems in manufacturing – an exploration of obstacles,” *Technology in Society*, vol. 78, p. 102648, 2024.
- [31] J. Walker, “Amr vs agv: A clear choice for flexible material handling,” <https://locusrobotics.com/blog/amr-vs-agv>, 14 Jul 2022, accessed: 03 Jul 2025.
- [32] S. A. H. Mohsan, N. Q. H. Othman, Y. Li *et al.*, “Unmanned aerial vehicles (uavs): practical aspects, applications, open challenges, security issues, and future trends,” *Intelligent Service Robotics*, vol. 16, no. 1, pp. 109–137, 2023.
- [33] C. Wang, S. Chai, T. Xu *et al.*, “Cp-rag: Mitigating distracting content in retrieval-augmented generation for industrial knowledge question answering,” *IEEE Internet of Things Journal*, vol. 13, no. 7, pp. 15 056–15 066, 2026.
- [34] C. Su, J. Wen, J. Kang *et al.*, “Hybrid rag-empowered multimodal llm for secure data management in internet of medical things: A diffusion-based contract approach,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 428–13 440, 2025.
- [35] R. Zhang, H. Du, Y. Liu *et al.*, “Interactive ai with retrieval-augmented generation for next generation networking,” *IEEE Network*, vol. 38, no. 6, pp. 414–424, 2024.
- [36] R. Kataishi, “Enhancing retrieval-augmented generation with topic-enriched embeddings: A hybrid approach integrating traditional nlp techniques,” *Natural Language Processing Journal*, vol. 14, p. 100200, 2026.
- [37] S. Prabhakaran, “Cosine similarity – understanding the math and how it works (with python codes),” <https://machinelearningplus.com/nlp/cosine-similarity/>, 2026, accessed: 19 Mar 2026.
- [38] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell *et al.*, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.
- [39] J. A. Heredia Álvaro and J. G. Barreda, “An advanced retrieval-augmented generation system for manufacturing quality control,” *Advanced Engineering Informatics*, vol. 64, p. 103007, 2025.
- [40] Y. Wang, Y. Wan, X. Lei *et al.*, “A retrieval augmented generation based optimization approach for medical knowledge understanding and reasoning in large language models,” *Array*, vol. 28, p. 100504, 2025.
- [41] N. Alabbasi, O. Erak, O. Alhussein *et al.*, “Teleoracle: Fine-tuned retrieval-augmented generation with long-context support for networks,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 170–13 182, 2025.
- [42] Q. Chen, L. Qin, J. Liu *et al.*, “Towards reasoning era: A survey of long chain-of-thought for reasoning large language models,” *arXiv:2503.09567*, 2025.
- [43] R. Y. Pang, K. J. K. Feng, S. Feng *et al.*, “Interactive reasoning: Visualizing and controlling chain-of-thought reasoning in large language models,” *arXiv:2506.23678*, 2025.
- [44] J. R. Chowdhury and C. Caragea, “Zero-shot verification-guided chain of thoughts,” *arXiv:2501.13122*, 2025.
- [45] S. Aggarwal, R. V. Mishra, and A. Awekar, “Evaluating chain-of-thought reasoning through reusability and verifiability,” *arXiv:2602.17544*, 2026.
- [46] R. Xu, W. Shi, Y. Zhuang *et al.*, “Collab-rag: Boosting retrieval-augmented generation for complex question answering via white-box and black-box llm collaboration,” *arXiv:2504.04915*, 2025.
- [47] T. Nguyen, P. Chin, and Y.-W. Tai, “Ma-rag: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning,” *arXiv:2505.20096*, 2025.
- [48] X. Wang, J. Wang, Y. Wang *et al.*, “Mars: toward more efficient multi-agent collaboration for llm reasoning,” *arXiv:2509.20502*, 2026.
- [49] Y. Li, W. Ke, J. Liu *et al.*, “Towards evidence-aware retrieval-augmented generation via self-corrective chain-of-thought,” *Information Processing & Management*, vol. 63, no. 2, Part A, p. 104369, 2026.
- [50] Y. Zhou, W. Yang, and Y. Gong, “Reinforcement learning with priority decentralized ppo for multi-vessel cooperative rescue scheduling in flood disaster,” *Alexandria Engineering Journal*, vol. 138, pp. 96–113, 2026.
- [51] H. Lee, J. Lee, I. Yeom *et al.*, “Panda: Reinforcement learning-based priority assignment for multi-processor real-time scheduling,” *IEEE Access*, vol. 8, pp. 185 583–185 583, 2020.
- [52] M. Fan, Y. Wu, T. Liao *et al.*, “Deep reinforcement learning for uav routing in the presence of multiple charging stations,” *IEEE Transactions on Vehicular Technology*, vol. 72, no. 5, pp. 5732–5746, 2023.
- [53] OpenAI, “text-embedding-ada-002,” <https://developers.openai.com/api/docs/models/text-embedding-ada-002>, 2026, accessed: 03 Apr 2026.
- [54] Chroma, “What chroma offers,” <https://docs.trychroma.com/docs/overview/introduction>, 2026, accessed: 03 Apr 2026.
- [55] OpenAI, “Gpt-5.4,” <https://developers.openai.com/api/docs/models/gpt-5.4>, 2026, accessed: 03 Apr 2026.
- [56] LangChain, “Langgraph overview,” <https://docs.langchain.com/oss/python/langgraph/overview>, 2026, accessed: 03 Apr 2026.
- [57] —, “Langchain overview,” <https://docs.langchain.com/oss/javascript/langchain/overview#langchain-overview>, 2026, accessed: 03 Apr 2026.